

Uppgift 3 – World Interfacing

Mål:

Dagens uppgift är att jobba med dataflöde till en AI, dvs. hur AI interface:ar med world data. Ni ska få prova på två vanliga tekniker för world interfacing: polling och events.

Generella krav:

- Det går bra att använda TGE, er egen motor eller ett tredje parts bibliotek (t.ex. raylib eller SDL).
- Programmet ska kompilera utan kompileringsvarningar på warning level 4 och med "treat warnings as errors" satt till "yes".
- Trådar som används ska joinas korrekt när programmet stängs av.
- Actors får inte flimra/hacka, deras movement ska vara smooth. (Inom rimliga gränser. Om en actor gör ett pyttelitet "hack" när den når sitt mål så är det ok, men den ska inte kontinuerligt bete sig glitchigt. Är du osäker om din movement är tillräckligt smooth så poke:a mig innan inlämning så gör jag en bedömning.)

Uppgift-specifika krav:

- Events som skickas ska vara one-shot events. Dvs om spelarkarakteren hackar en dator så ska ni inte skicka ett "PlayerIsHacking"-event varje frame, utan det ska bara skickas en gång när spelaren börjar hacka, alltså t.ex. ett "PlayerStartedHacking"-event. Annars blir det lite att man använder events som polling, vilket går emot idén med events.
- Getters i AIPollingStation ska kolla om de har blivit call:ade tidigare samma frame. Om de inte har det så ska de uppdatera värdet de returnerar innan de returnerar det.

Uppgiften:

- När spelet börjar ska man se fem actors och ytterliggare tre statiska entities.
- En actor är spelaren. Den ska styras av en PlayerController eller ArriveController som reagerar på mouse input. Spelaren ska röra sig dit man senast klickade.
- Våra fyra andra actors är robotvakter som ska upptäcka när spelaren hackar datorer.
- Våra tre statiska entities är datorer som spelaren kan hacka. Spelaren "hackar" dem genom att bara befinna sig inom ett visst avstånd från dem.
- Det finns två "smarta" vakter och två "dumma" vakter.
 - De smarta vakterna rör sig mot den senaste datorn som spelaren har försökt hacka och stannar inte förrän de är framme vid den, oavsett om spelaren fortfarande är där eller inte.
 - De dumma vakterna rör sig bara mot den datorn som spelaren just nu hackar. Om spelaren inte hackar någon dator just nu så står de stilla.
- En av de smarta vakterna ska styras med hjälp av polling och den andra med hjälp av events.

- En av de dumma vakterna ska styras med hjälp av polling och den andra med hjälp av events.
- Vi kommer alltså ha fyra olika controllers:
 - SmartGuardPollController
 - SmartGuardEventController
 - StupidGuardPollController
 - StupidGuardEventController.
- De som använder polling ska göra det via en AIPollingStation.
- De som använder events ska göra det via en AEventManager.
- Alla vakter ska starta på olika positioner, men när vakter med samma beteende överlappar så kommer de troligen inte gå ifrån varandra igen eftersom de har samma beteende. Detta är ok, och t.o.m. bra. Då kan vi verifiera att vi kan uppnå precis samma beteende med olika metoder.
- Sprites för spelare, vakter och datorer finns på Omniway, men det går såklart bra att använda egen art.
- Endast spelare, vakter och datorer behöver visualiseras. Frivilligt att rita ut steering force och desired velocity.

Inlämning:

- På Omniway. Parprogrammering går fint, skriv i så fall i er inlämning vem ni har parprogrammerat med.
- En mapp för Exe och en för Source.
- Ni ska lämna in antingen en Release-exe, eller en Debug-exe och en Retail-exe. Detta för att debug drawing är disabled i Retail.
- Inga skräpfiler som .vs-mappar eller liknande får förekomma. Var noga med att köra Clean på din solution innan du lämnar in.